

NODE WAR - Master Design Document v2.1

NODE WAR - MASTER DESIGN DOCUMENT

Version 2.1 | August 2026

Status: Phase 8 networking verified. Architectural restructure planned next.

Development target: Playable shareable build in approximately 3 months.

=====

PART I: THE GAME

=====

WHAT IS NODE WAR

Node War is a real-time 1v1 strategy game played on a node graph. Players command villagers to claim territory, gather resources, build armies, and breach the enemy core.

The entire game revolves around one resource: villagers. Every villager farming is not fighting. Every villager claiming is not defending. Every villager lost costs time or economy.

WIN CONDITION

Three enemy units reach your core node. Each breach permanently consumes the attacking villager. Winning requires sacrificing three units. Defense is about having units present to intercept.

CORE TENSION

Players must split villagers across competing priorities: expansion (claiming nodes), economy (farming, mining, smelting), military (equipping soldiers), defense (protecting the core), and aggression (pushing toward the enemy). Every decision involves opportunity cost.

THE DRAFT LAYER (future, core to long-term design)

Beyond the base board, each player brings a small selection of additional node types into the

match. These populate their side of the board and meaningfully alter the strategic landscape. This system drives replayability, progression, skill expression, and long-term engagement. See Part IX for detail.

MATCH FEEL

Fast, tense, constant decisions. No idle periods. Target match duration: 3-8 minutes. The small villager count means every unit matters. The game rewards reading the opponent, efficient resource allocation, timing pushes, and adapting to board state.

PLATFORM INTENT

Development: Windows PC (current)

Target release: Mobile (iOS/Android) primary, PC (Steam) secondary

Design principle: One simulation architecture. Platform-specific input, UI, and presentation shells only.

Timeline: Playable shareable build in approximately 3 months. Short milestones preferred over long plans.

=====

PART II: ARCHITECTURE OVERVIEW

=====

DESIGN PHILOSOPHY

The simulation is the game. Everything else is presentation or transport.

Game rules exist in exactly one place. The simulation never knows how inputs arrive or how results are displayed. Adding networking, AI, replays, or new platforms never requires changing game logic. The game is deterministic: same inputs always produce the same results on any machine.

LAYER ARCHITECTURE

SIMULATION LAYER

Owns: all gameplay state, rules, timers, combat, production, claiming, win conditions

Receives: commands as data. Produces: updated state as data.

Constraints: integers only, no platform APIs, deterministic, no unseeded randomness

INPUT LAYER

Owns: translating human, bot, or network actions into commands

Produces: GameCommand structs written to InputBuffer

Constraint: never modifies simulation state directly

COMMAND LAYER

Owns: validating and applying commands to simulation state

Order contract: P0 commands first (in issue order), P1 commands second, then SimulateTick

TICK LAYER

Owns: timing, accumulation, calling SimulateTick at fixed 10hz rate

Provides: TickAlpha for visual interpolation between ticks

Variants: TickRunner (local/bot), LockstepRunner (networked)

NETWORKING LAYER

Owns: transporting commands between machines, connection lifecycle

Never touches game logic. Only moves input data.

VIEW LAYER

Reads SimulationState every frame. Writes nothing to it.

Uses floats, interpolation, Unity APIs, Time.deltaTime freely.

UI LAYER

Reads SimulationState for display.

Writes commands to InputBuffer via player interactions.

SCENE/LIFECYCLE LAYER

Owns: application flow, scene transitions, match setup and teardown.

Manages persistent objects across scenes.

PERSISTENCE LAYER (future)

Owns: local settings, player data, server synchronization.

INFORMATION FLOW

Human/Bot/Network Input

|

v

InputBuffer (per-player command queue)

|

v

CommandProcessor (validate, apply to state)

|

v

GameSimulation.SimulateTick (deterministic rules)

|

v

SimulationState (single source of truth)

|

v (read only)

View Layer and UI Layer

For networked play: both machines run the identical simulation from identical inputs. Only input data travels over the network. No game state is ever synchronized directly.

DETERMINISM CONTRACT

The simulation guarantees identical results on any machine given identical inputs:

- All simulation values are integers (no floating point drift)
- Fixed 10hz tick rate regardless of frame rate
- Command processing order is fixed and enforced
- No platform-dependent APIs inside simulation
- No randomness without deterministic seeded RNG stored in SimulationState
- Arrays used for all simulation collections (no Dictionary or HashSet iteration)
- All sorting has total-order comparators with ID tiebreakers

This enables: lockstep networking, replay recording, bot testing, and deterministic bug reproduction.

ANTI-CHEAT NOTE

In a lockstep game, a modified client produces different simulation state than an honest client. Desync detection (hash comparison every 50 ticks) catches this within 5 seconds. No additional anti-cheat infrastructure is needed.

MULTIPLE SIMULTANEOUS MATCHES

Multiple pairs of players can play simultaneously. Each match is a completely independent P2P connection. Ten matches can run across ten pairs of machines with no shared coordination. This is a free property of the architecture.

=====

PART III: CURRENT PROJECT STATE

=====

IMPLEMENTED AND VERIFIED

Simulation: 10hz fixed tick, 28-node grid board, Dijkstra pathfinding with ownership preference multipliers, movement with per-edge travel time, combat with deterministic targeting and round-robin assignment, claiming with 4x decrement, bonus villager spawning, per-villager production timers (Farm/Mine/Forge), Forge material allocation, Equip and Respawn commands with resource validation, passive healing every 30 ticks, breach detection, win condition (3 breaches), 25-villager cap per player, full state hasher for desync detection.

Input: Circle/lasso drag selection, shift+click additive selection, right-click move commands, E equip, R respawn, per-player routing, UI click-through prevention.

View: Node coloring by ownership, villager slot positioning per state, movement interpolation via TickAlpha, selection highlighting, billboard sprites.

UI: HUD resource wheels with DOTween animation, breach bars, villager count, node panel with district-specific content (production disks, forge allocation wheel, barracks equip list, core respawn list).

Networking: Raw UDP, deterministic InputSerializer, NetworkManager with background receive

thread, LockstepRunner with 2-tick input delay, stall-on-missing, resend, disconnect detection, desync hash comparison. LobbyUI with host/join/handshake. ITickProvider interface shared by TickRunner and LockstepRunner.

Verified working: Two standalone builds connect via localhost. Lockstep advances in sync. 15 consecutive hash checks with zero desync. Disconnect detected within 2 seconds.

Required for any build: Application.runInBackground = true. Windowed build configuration.

KNOWN BUGS (ACTIVE)

1. VillagerView receives null tickProvider in network mode. Villagers snap instead of interpolating. Fix: initialization order restructure (Phase A).
2. Pathfinding cost collapse with defaultEdgeWeight=3. Fix: set to 4 in Inspector.
3. Camera defaults to P1 side. Cosmetic.

PROTOTYPE-ONLY (expected to change)

OnGUI lobby, shared InputBuffer for local debug, Tab-key player switching, hardcoded board, all initialization in Awake, programmer-art sprites, no audio, no animation beyond position interpolation.

STABLE LONG-TERM

SimulationState as single truth, SimulateTick as pure deterministic function, integer simulation math, fixed tick rate, InputBuffer/GameCommand pattern, P0-first command order, view as read-only consumer, lockstep P2P for networked play.

=====

PART IV: GAME SYSTEMS DETAIL

=====

THE BOARD

A graph of nodes connected by edges. Current board: 28 nodes, 4x7 grid, orthogonal connections.

District types: Core (starting base, breach target, respawn point), Farm (produces Food), Mine (produces Materials), Forge (converts Materials to Metal), Barracks (soldier equip location), Village (grants bonus villagers on claim), None (empty connectors).

Pathfinding prefers friendly territory via ownership-weighted edge costs.

VILLAGERS

The only unit type. Role is determined by context. Farmers on owned farms, miners on owned mines, smelters on owned forges, soldiers when equipped at barracks, claimers on neutral or enemy nodes, fighters when sharing a node with enemies.

Soldiers: higher attack damage, slower movement, permanent until death.

Workers: lower damage but resist in numbers.

RESOURCES

Food: produced by Farmers. Spent on soldiers (2) and instant respawns (1).

Materials: produced by Miners. Spent on soldiers (1) and forge conversion.

Metal: produced by Smelters (costs 1 Material per cycle). Future advanced uses.

All resources are player-global. A worker on any owned farm contributes to the total regardless of board position.

COMBAT

Automatic when enemies share a node. Deterministic round-robin targeting sorted by priority descending, ID ascending. Per-unit cooldowns. Death triggers respawn timer (50 ticks, 5 seconds) or instant respawn for 1 Food.

Design intent: soldiers kill efficiently but cost resources. Workers resist in numbers. Forcing meaningful tradeoffs.

CLAIMING

Single integer bar per node (-10000 to +10000). Players push toward their side. Decrementing enemy progress is 4x faster than building your own. Completion transfers ownership and spawns bonus villagers at Village nodes. Combat freezes claiming.

WIN CONDITION

Three breaches. Each breach permanently consumes the attacking villager. Defense requires stationing units at or near the core. The core has no built-in defenses.

PART V: GAME FLOW AND SCENE ARCHITECTURE

INTENDED FLOW

Home Screen

|
v

Play: Matchmaking / Local / Bot Match

|
v

Lobby: Match Found, Countdown

|
v

Gameplay Scene

|
v

Game Over: Results, Trophy Update

|
v

Rematch or Return Home

SCENE STRUCTURE

Scene 1 (Home/Lobby): Main menu, player profile, matchmaking entry, settings, bot match entry. Network connection handling.

Persistent (DontDestroyOnLoad): MatchConnection (live network socket, localPlayerID, matchSeed). PlayerProfile (settings, cached progression). AudioManager (music continuity).

Scene 2 (Gameplay): SimulationState created fresh each match. All views, HUD, camera, LockstepRunner or TickRunner. Everything destroyed on scene unload.

CORRECT INITIALIZATION ORDER (resolves VillagerView bug)

1. MatchConnection exists with full config
2. GameManager.Awake() reads MatchConnection
3. InitializeNodes() from seed or hardcoded
4. InitializePlayers(), InitializeVillagers()
5. InitializeInputSystems()
6. Create TickProvider (LockstepRunner or TickRunner)
7. SpawnNodeViews(), SpawnVillagerViews() (valid tickProvider now exists)
8. InitializeUI()
9. Optional countdown
10. Ticking begins next frame

All views receive a valid tickProvider because the runner is created before views are spawned.

MATCH END FLOW

1. SimulateTick sets gameOver flag
2. View shows victory/defeat presentation
3. Results and trophy change displayed
4. Rematch: reload gameplay scene, same connection, new seed
5. Return home: close socket, destroy MatchConnection, load Scene 1

ERROR STATES

Opponent disconnects: 10-second reconnect window (see Connectivity section).

Lobby timeout: error message, remain in lobby.

Version mismatch: detected via handshake metadata, prevent match start.

Desync detected: log tick and hash values, end match, show error.

=====

PART VI: CONNECTIVITY AND DISCONNECT HANDLING

=====

DISCONNECT BEHAVIOR (three-phase)

Phase 1 - Automatic hold (0 to 10 seconds):

Both machines stall naturally. Lockstep already does this when inputs stop arriving. Both screens show "Opponent disconnected. Waiting..." with countdown. Neither player can act.

Phase 2 - Opponent choice (10 to 60 seconds):

The waiting player sees two options: continue waiting, or end the match. Ending awards the waiting player a forfeit win.

Phase 3 - Hard forfeit (60 seconds):

If the disconnected player has not returned, match ends automatically. Disconnected player forfeits.

Reconnect flow (designed, not yet implemented):

Disconnected player relaunches and rejoins using session ID stored in MatchConnection. Waiting machine recognizes the session ID and resumes. Both machines were stalled at the same tick, so state is identical. Resume from exact same point.

Trophy consideration (future): verified connection loss produces reduced trophy penalty versus intentional disconnection.

BACKGROUNDING ON MOBILE

Same tiered behavior as disconnect: 10 seconds automatic, then opponent choice, hard forfeit at 60 seconds. Player who backgrounded too long sees "You were away too long" on return if match ended.

NETWORK ENVIRONMENT CONSIDERATIONS

Same-room WiFi: 1-5ms, near-zero packet loss. No special handling.

Managed networks (EDUROAM and similar): Block peer-to-peer UDP between devices on the same subnet. Direct P2P will not work. Relay service required.

Internet: Requires NAT traversal or relay. Direct P2P needs port forwarding. Relay handles this automatically.

RELAY SERVICE PLAN

Primary: Unity Gaming Services Relay (UGS).

No platform dependency. Works on mobile and PC. Free tier for prototyping. Native Unity integration. Does not require Steam.

Secondary: Steam Relay.

Available only when distributing via Steam. Excellent infrastructure. Not suitable as primary option for mobile-first release.

When relay is implemented: only NetworkManager changes. LockstepRunner, InputSerializer, GameSimulation, and all game logic remain identical. The relay is a packet forwarding intermediary, invisible to game logic.

Timing: Implement after LAN play is fully working and tested. Required before EDUROAM or internet play is practical.

=====

PART VII: LOCAL BOT AND SINGLE-PLAYER

=====

PHASE 1 - PRACTICE BOT (after Phase A)

A bot is another source of commands. It reads SimulationState each tick and produces GameCommand arrays. The simulation is identical whether the opponent is human or bot. TickRunner is used, no networking needed.

Heuristic bot behavior (200-300 lines):

- Send spare villagers to claim nearest unowned nodes
- Equip soldiers when resources allow and a barracks is owned
- Send soldiers toward enemy territory when army exceeds threshold
- Keep at least one unit near own core

This is sufficient for solo testing, practice, balance iteration, and demonstrating the game on one machine.

PHASE 2 - NAMED PROGRESSION OPPONENTS (future)

A series of named bot opponents with distinct personalities and difficulty levels. Each personality emphasizes a different strategic approach: rusher, fortifier, expansionist, tactician. Difficulty is controlled by decision speed and target selection quality. Not machine learning. Not search-based. Parameterized heuristic behavior tuned for each opponent.

Tied to arena progression: each arena introduces bot opponents appropriate to that tier. Beating them is how players practice before facing real opponents.

BOT AS ARCHITECTURAL VALIDATION

The bot's architectural placement (reads state, produces commands, slots into the same interface as a human) validates that the command-layer separation is working correctly. If the simulation behaves identically whether inputs come from a human, a bot, or a network, the architecture is correct.

=====

PART VIII: ARENA SYSTEM AND THE DRAFT

=====

OVERVIEW

The arena system gives matches long-term strategic depth beyond individual games. Players progress through arenas by earning trophies. Each arena unlocks a tier of more powerful or specialized node types. Players bring a selection of these into matches, shaping the board and defining their strategy.

This is the primary driver of replayability and the long-term identity of the game.

THE DRAFT

Before each match:

1. Base board generates normally (seeded, symmetric)
2. Each player selects 3 additional node types from their unlocked pool
3. These 3 types populate designated draft slots on their side of the board
4. Both players can see each other's selections before the match begins
5. Match starts with a board shaped by both players' strategic choices

This introduces a metagame layer: certain node combinations are powerful, some counter others. Players develop strategies around preferred node types. Choosing a complementary set is a skill separate from in-match play.

ARENA TIERS (proposed, subject to design iteration)

Arena 1 - ANCIENT

Primitive warfare. Base node types only. Designed for learning core mechanics.

Theme: earthen walls, hunting grounds, tribal gathering sites.

Node tier: basic variations on existing types (slightly faster Farm, slightly stronger Barracks).

Arena 2 - BRONZE AGE

First specialized military and economic nodes.

Theme: bronze smithies, fortified outposts, granary towers.

Node tier: conditional production (node produces more when adjacent conditions are met), basic buff nodes.

Arena 3 - STEAM ERA

Industrial output and mechanized military.

Theme: steam foundry, rail depot, armory.

Node tier: high-output delayed production, chain production (one node feeds another), mechanical troop options.

Arena 4 - SORCERY / PRAYER

Mystical, high-risk-high-reward mechanics.

Theme: ritual circle, leyline convergence, oracle tower.

Node tier: nodes with special claiming effects, nodes that affect breach mechanics, advanced resource conversion.

Additional arenas added as the game matures. Each arena is a content addition, not an architectural change.

BOARD GENERATION WITH DRAFT

Seed determines base board topology. Player draft determines district types of draft slots on each side. Match config packet carries: seed, P0 draft selection, P1 draft selection. Both machines generate identical boards from the same inputs. No randomness at match time. Board is fully determined before ticking begins.

THE LOSING EXPERIENCE

Because players build their own board through drafting, the cause of a loss is visible: the opponent's node combination outperformed yours, or you misplayed the board you built. Post-game shows the board, final state, and key statistics. A player should be able to identify what went wrong without a replay system.

PART IX: TROPHY AND PROGRESSION

TROPHY CHANGES

Win: gain trophies (amount scales with opponent's relative trophy count).

Loss: lose trophies (reduced loss against stronger opponents).

Arena threshold: reaching a trophy count unlocks the next arena and its node tier.

COMPETITIVE INTEGRITY

Trophy count is server-authoritative (future). Locally cached for display. Cannot be self-reported or locally modified.

Disconnect: server distinguishes verified connection loss (reduced penalty) from repeated rage-quit pattern (full penalty).

Version mismatch: clients on different versions are blocked from matching. Version checked during handshake.

=====

PART X: MATCHMAKING

=====

PROTOTYPE (current)

Manual IP entry. Host and joiner on same network. No accounts, no trophies, no server.

LAN DISCOVERY (near-term, after Phase A)

Host broadcasts presence via UDP on a discovery port every 1 second. Joiner listens and displays available games. Tapping a game initiates normal handshake. No server required. Does not work on managed networks.

INTERNET PLAY VIA RELAY (after LAN is stable)

Routes packets through relay servers. Bypasses NAT and managed network restrictions. Required for EDUROAM, internet play, and mobile cellular play. Transport change only. All game logic identical.

PRODUCTION MATCHMAKING (future, requires backend)

1. Player enters queue with server-stored trophy count
2. Server finds opponent within trophy range (expanding over time)
3. Server connects both via relay
4. Match runs peer-to-peer through relay
5. Both clients report result to server
6. Server validates (both reports agree), updates trophies

A backend server becomes necessary when: strangers need to find each other, competitive trophies must be tamper-proof, or player base requires more than manual connection.

=====

PART XI: MONETIZATION (FUTURE, NOT CURRENT SCOPE)

=====

APPROACH

Free to play. No pay-to-win. Competitive integrity is non-negotiable.

PURCHASABLE

Cosmetic node variants: visual reskins with no gameplay difference.

Cosmetic villager variants: appearance changes, no stat effects.

Accelerated progression: purchase access to node types that are otherwise earned through play. The ceiling is identical. Purchases compress timeline only.

Optional ads: opt-in for small practice-mode bonuses. Never forced. Never during a match.

NOT PURCHASABLE

Node types stronger than free equivalents. Villager stats. Trophy counts. Match outcomes.

Anything affecting competitive results.

=====

PART XII: PERSISTENCE AND PLAYER PROFILE

=====

LOCALLY PERSISTED (no server required)

Settings, tutorial progress, local bot match history, cached trophy count (display only), unlocked node types (mirrored from server, readable offline), preferences. Saved on match completion, settings change, and app suspension.

SERVER-AUTHORITATIVE (future)

Competitive trophies, verified match history, arena progression, account identity.

MOBILE CONSIDERATIONS

Save on suspension, not only on clean exit. Offline bot play works without connection.

Competitive queue requires connection. Account tied to platform identity (Apple ID, Google Play) when introduced. Local data is cache, server is truth.

=====

PART XIII: CAMERA DESIGN

=====

CURRENT

Orthographic, approximately 45-60 degrees, middle-mouse pan, scroll zoom. Functional.

INTENDED

Perspective camera with strategy-game controls.

Hierarchy: parent object holds world position (pan target), child pivot holds viewing angle, camera component on child uses perspective projection.

Zoom moves camera closer or farther from board. FOV stays constant. Pan speed scales with zoom distance. Board fully visible at maximum zoom-out on all aspect ratios.

Platform input: Desktop: middle-mouse drag and scroll. Mobile: two-finger drag and pinch zoom. Controller: right stick and triggers.

Touch disambiguation: finger starting near a unit is selection intent, finger starting on empty ground with no selection is pan, two fingers always pan/zoom.

=====

PART XIV: PRESENTATION

=====

STYLE

Paper Mario-inspired 2.5D. Flat sprites in 3D space. Bold outlines, limited palettes, clear silhouettes. Costume overlays per suit type.

NORMAL MAPS

Provide simulated depth and dynamic lighting response on flat sprites. URP Sprite-Lit material supports this natively. Medium priority. Add after base art exists. High visual benefit for moderate art effort.

SELECTION FEEDBACK

Selection ring sprite beneath unit (immediate, functional) plus outline shader as a later polish pass. Both can coexist.

ANIMATION PRIORITY ORDER

1. Gameplay clarity: state color coding, death signal, damage flash, command acknowledgment.
2. Gameplay communication: breach event presentation, claim completion, resource indicators.
3. Character life: idle bounce, work animation, walk hop, attack, death crumple.
4. Environmental polish: ambient node animations, particles.

ATTENTION AND POINTS OF INTEREST

Off-screen event indicators. Lives in view/UI layer. Reads SimulationState, writes nothing.

Priority order: enemy unit near own core, friendly villager dying, friendly node being lost, enemy claiming a node, production milestone. Edge-of-screen arrows, color-coded by severity, tappable to snap camera. Maximum 3-4 simultaneous. Per-event cooldown.

=====

PART XV: COMMERCIAL SYSTEMS

=====

CRITICAL (players notice their absence immediately)

1. Tutorial and onboarding: new players quit without this
2. Match countdown (3-2-1-GO): synchronized start, fairness signal
3. Loading and transition screens: blank screens feel broken
4. Connection quality indicator: player needs latency warning before disconnect
5. Disconnect handling with clear messaging and outcome

HIGH IMPORTANCE

6. Audio feedback: silent games feel unresponsive
7. First-time user experience: guided first match, progressive mechanic disclosure
8. Error messaging with recovery options
9. Match history: players want to review results
10. Ready state before match: both players confirm

MEDIUM

11. Forfeit option: alternative to disconnecting mid-match
12. Queue cancellation: must be able to back out before match starts
13. AFK detection: extended command silence should trigger warning
14. Version mismatch detection and messaging
15. Matchmaking timeout messaging

SUBTLE SYSTEMS OFTEN MISSED

Input buffering during countdown (commands before GO must be discarded). Session identity for reconnection. Progressive tutorial (one mechanic per session, not everything at once). Post-match clarity (player understands cause of loss from results screen).

=====

PART XVI: PLATFORM DESIGN

=====

PRINCIPLE

One simulation. Platform-specific shells for input, camera, and presentation only.

DESKTOP

Mouse and keyboard as currently implemented. Higher information density acceptable.

MOBILE (primary release target)

Input: tap select, drag lasso, tap node for move command, two-finger pan, pinch zoom.

UI: 44pt minimum touch targets, bottom-anchored primary actions, safe areas respected.

Performance: 30fps minimum on mid-range devices, battery saver mode, reduce effects under thermal pressure.

App lifecycle: suspend grace period per Connectivity section. Save on suspension.

CONSOLE (future)

Virtual cursor via left stick. Face buttons for actions. Right stick pan, triggers zoom. TV-safe areas, larger text. No architectural changes needed. Add when targeting.

=====

PART XVII: SETTINGS

=====

Audio: master, music, SFX, UI sounds.

Graphics: quality preset, particles, frame rate target.

Gameplay: camera speed, zoom sensitivity, camera momentum, confirm-on-command option.

Accessibility: colorblind mode, increased UI scale, reduced motion.

Mobile: haptics, battery saver.

All settings stored locally. Applied on launch. No server dependency.

=====

PART XVIII: ARCHITECTURAL BOUNDARIES

=====

SIMULATION owns: SimulationState, GameSimulation, CommandProcessor, Pathfinding, SimulationStateHasher, all gameplay constants and rules.

INPUT owns: InputBuffer, SelectionSystem, CommandSystem, bot decision functions.

NETWORKING owns: NetworkManager, InputSerializer, LockstepRunner, LobbyUI, relay transport (future), LAN discovery (future).

VIEW owns: VillagerView, NodeView, NodeSlotManager, Billboard, NodeHighlight, all particles and animation.

UI owns: HUDManager, all panel content, selection lasso visual, attention indicators, all menus and settings screens.

LIFECYCLE owns: scene management, MatchConnection, application initialization, DontDestroyOnLoad objects.

PERSISTENCE owns: local save/load, settings, profile cache, server sync (future).

RULE: Information flows one direction only. Simulation never reads from view. View never writes to simulation.

ADDING A NEW FEATURE - CHECKLIST

1. Does it affect game state? Add integer fields to SimulationState. Add to hasher.
2. Does a player trigger it? New CommandType. New case in CommandProcessor. Verify GameCommand struct is sufficient or update struct and InputSerializer together.
3. Does it involve randomness? Deterministic seeded RNG in SimulationState, advanced only in SimulateTick, included in hash.
4. Does it iterate collections? Arrays or List only. No Dictionary or HashSet in simulation.
5. Does it change array sizes? Follow SpawnBonusVillagers pattern. GameManager.Update() detects length change and spawns views.
6. Is it purely visual? View layer only. Floats and Unity APIs fine. Never write to SimulationState.

PART XIX: DEVELOPMENT ROADMAP

PHASE A - FOUNDATION (immediate)

Purpose: Correct application lifecycle. Fix architectural debt before it compounds.

Systems: Two-scene architecture, MatchConnection with DontDestroyOnLoad, correct initialization order (eliminates VillagerView null tickProvider bug), return to lobby on disconnect and game-over, windowed build, Application.runInBackground.

Result: Game starts, plays, ends, and restarts cleanly. Network matches work without manual workarounds.

Testing: Phase 8 playtest checklist with scene transitions. Disconnect and game-over both return to lobby.

PHASE B - LOCAL BOT

Purpose: Enable solo testing and practice without a second machine.

Systems: BotPlayer as heuristic command source, bot match flow bypassing lobby, TickRunner used (no networking), simple difficulty parameter.

Result: Full matches playable solo. Can demonstrate game on one machine. Can iterate on balance without a network partner.

Testing: Play 10 matches against bot. Verify recognizable decisions. Verify no simulation errors.

PHASE C - CORE PRESENTATION

Purpose: Make the game feel like a real game to an outside observer.

Systems: Proper sprites with suit costumes, basic animations (idle, walk, attack, death), audio (music and SFX), match countdown, proper victory/defeat screen, resource gain indicators, breach event presentation.

Result: Visually and audibly engaging. Suitable for sharing with people unfamiliar with the project.

Testing: Show to someone unfamiliar. Do they understand what is happening?

PHASE D - LAN DISCOVERY AND RELAY

Purpose: Players find each other without typing IP addresses. Works on EDUROAM and across the internet.

Systems: UDP broadcast LAN discovery, Unity Relay integration, connection quality indicator, improved disconnect messaging, session identity for reconnect attempt.

Result: Click Play on two machines on the same network. Match starts. Works on managed networks.

Testing: LAN discovery on same WiFi. Relay play across different networks.

PHASE E - PROCEDURAL BOARDS AND DRAFT

Purpose: Every match is unique. Core replayability and strategic variety.

Systems: Seeded board generator, draft UI (select 3 node types before match), seed and draft exchange in match config packet, board validation for connectivity and fairness.

Result: Players shape the board before the match. No two games play the same. Foundation for arena progression.

Testing: Verify identical board generation from same inputs on both machines. Play varied draft combinations.

PHASE F - ARENA AND PROGRESSION

Purpose: Wins feel meaningful. Long-term motivation established.

Systems: Trophy gain and loss, arena unlock thresholds, new node types per arena tier (Ancient through Sorcery), trophy display, arena presentation, bot opponents per arena.

Result: Winning earns progression. New content unlocks. Competitive motivation exists.

Testing: Verify trophy math. Verify node unlock gating. Play through first two arenas.

PHASE G - TUTORIAL AND ONBOARDING

Purpose: New players can learn without external instruction.

Systems: Scripted tutorial against passive bot, progressive mechanic introduction, highlight system, tutorial progress tracking, contextual hints in early matches.

Result: Unfamiliar player understands the game in under 5 minutes.

Testing: Watch someone unfamiliar play the tutorial without explanation. Can they complete a real match afterward?

PHASE H - MOBILE BUILD

Purpose: Primary release platform working.

Systems: Touch input (tap, drag, pinch, two-finger pan), mobile UI adaptation (bottom sheet, safe areas, touch targets), performance profiling, battery saver, app lifecycle handling.

Result: Full game playable on iOS and Android.

Testing: Device testing on iOS and Android. Multiple aspect ratios. Mid-range device performance.

=====

PART XX: DEFERRED / DO NOT BUILD YET

=====

DEFERRED UNTIL AFTER PHASE D (relay working):

- Production internet matchmaking with backend server
- Account system and cross-device progression
- Server-authoritative trophies

DEFERRED UNTIL AFTER PHASE F (arenas exist):

- Season structure and resets
- Cosmetic unlock system
- Accelerated progression purchases (nothing to accelerate before progression exists)

DEFERRED UNTIL CORE GAME IS PROVEN FUN:

- Additional unit types (Tank, Swift variants)
- Advanced economy (disconnected node penalties, trade routes)
- Fog of war
- Wall construction
- 4-player mode (breaks lockstep assumptions, requires architectural changes)
- Complex search-based AI (heuristic bot is sufficient)

DEFERRED INDEFINITELY (outside current vision):

- Full 3D character models
- Dedicated authoritative game server
- User-generated content
- Replay sharing
- Spectator mode

WHY THESE ARE DEFERRED

Every item deferred either depends on something not yet built, adds complexity before the core

is proven, or solves a problem the current scale does not have yet. Short milestones over long plans. Every system built should serve the current goal: a playable, shareable build that demonstrates the game concept convincingly.

=====

END OF DOCUMENT

=====